UNIVERSIDAD DEL VALLE DE GUATEMALA
Sistemas Operativos
Sección – 10
Ing. Kimberly Barrera



Proyecto 2

Leonardo Dufrey Mejía Mejía
María José Girón Isidro
Cindy Mishelle Gualim Perez

GUATEMALA, 10 de agosto de 2026

Índice

1. Introducción 3
2. Antecedentes 3
3. Punto de partida: Laboratorio 03 3
4. Diseño de las pruebas 4
5. Estrategia base: distribución cíclica 5
6. Estrategia alternativa: bloques intercalados (dovetailing) 6
7. Ventajas, limitaciones y discusión 9
8. Conclusiones y recomendaciones 10
9. Referencias 11
Anexo 1 — Catálogo de funciones y librerías 14
Anexo 2 — Bitácora de pruebas y Speedup 17

Proyecto 2

1. Introducción

Comparamos la distribución cíclica del Laboratorio 03 con una estrategia de bloques intercalados (dovetailing) para buscar por fuerza bruta una clave AES-128 con Open MPI y 4 procesos. El objetivo fue reducir el tiempo de búsqueda de las claves del enunciado y explicar por qué ese tiempo depende de la posición de la clave dentro del recorrido de cada proceso.

Este informe presenta lo que se implementó, cómo se probó y los resultados obtenidos, incluyendo la comparación de las estrategias de distribución y el análisis de Speedup.

2. Antecedentes

- **AES-128.** Cifrado simétrico por bloques de 16 bytes con una clave de 128 bits, estandarizado por NIST en FIPS 197. El mismo valor de clave sirve para cifrar y descifrar.
- **Modo CBC.** Cada bloque se combina con el bloque cifrado anterior antes de cifrarse; el primero usa un vector de inicialización (IV). Lo usamos en lugar de GCM porque el enunciado pide validar cada candidata con una frase de búsqueda, no con una etiqueta de autenticación.
- **Fuerza bruta.** Probar claves candidatas hasta que el texto descifrado contenga "es una prueba de". El trabajo crece con el tamaño del espacio: 2^56 claves en este proyecto, frente a 2^128 en AES real, que es inalcanzable.
- **Open MPI.** Implementación del estándar MPI para memoria distribuida: cada proceso tiene su propia memoria y se coordinan con mensajes (MPI_Bcast, MPI_Send, MPI_Irecv, MPI_Reduce).

3. Punto de partida: Laboratorio 03

Seguimos en C con OpenSSL (interfaz EVP) y Open MPI, reutilizando make_key(), parse_u64(), fail(), la estructura MPI, el Makefile y medir_speedup.sh. Los cambios realizados fueron estos:

Aspecto	Laboratorio 03	Proyecto 02
Modo de cifrado	AES-128-GCM, valida con etiqueta	AES-128-CBC, valida con la frase "es una prueba de"
Mensaje	Parámetro -m, máximo 256 bytes	Archivo .txt con validaciones

Espacio de claves	Hasta 2^40	2^56
Estrategias	Solo cíclica	-s ciclica, -s bloques, -s cantor con -K
Terminación	MPI_Allreduce cada 512 claves	Aviso asíncrono con MPI_Irecv y MPI_Test
Búsquedas largas	Sin límite	Opción -t con estimación del tiempo total

La conversión de clave se mantiene: el número se coloca en los últimos 8 bytes de la clave y los primeros 8 quedan en cero.

4. Diseño de las pruebas

Todas las ejecuciones paralelas usaron mpirun -np 4. El tiempo se midió con MPI_Wtime desde la sincronización inicial hasta que terminó el último proceso, y se reportó el promedio de varias ejecuciones.

Archivos de entrada

Archivo	Contenido	Uso
datos/mensaje.txt	"Esta es una prueba de proyecto 2" (32 bytes, sin salto de línea final)	Búsqueda, incisos 2 y 3
datos/texto_largo.txt	Párrafo de varias líneas con tildes y signos	Demostración de cifrado y descifrado, inciso 1
datos/vacio.txt y un nombre inexistente	Vacío / no existe	Programación defensiva

El archivo de búsqueda se crea con printf "Esta es una prueba de proyecto 2" > datos/mensaje.txt para evitar el salto de línea que agregan los editores.

Claves de prueba

Prueba	Clave	Propósito
a)	123456	Clave pequeña, del enunciado
b)	18014398509481984 (2^54)	Inicio exacto del bloque del proceso 1
c)	18014398509481985 (2^54 + 1)	Segunda clave del bloque del proceso 1
Extra	3377699720527877 (3·2^50 + 5)	Clave a mitad de bloque, favorable al dovetailing
Extra	18014398509481983 (2^54 − 1)	Peor caso de las estrategias por bloques

Mediciones

1. Versión secuencial con la clave 123456, para calcular el speedup: S = T_secuencial / T_paralelo.
2. Cada clave con las estrategias cíclica, bloques (K = 1) y dovetailing (K = 16 y 256).
3. Las búsquedas que no terminan se cortaron con -t. El programa reporta claves por segundo y estima el tiempo total a partir de la posición de la clave en el recorrido de su proceso.
4. Cada ejecución quedó registrada en la bitácora con comando, salida y captura, incluida la compilación.

5. Estrategia base: distribución cíclica

Con la estrategia cíclica, la clave k le toca al proceso k mod p y se prueba en su intento k div p. Es la distribución del Laboratorio 03: el proceso r prueba r, r + 4, r + 8, …

```
para i = rank; i < 2^56; i += p:
   si contiene_frase(descifrar(i)): encontrada_local = i
   cada 512 intentos: MPI_Allreduce(encontrada_local, MIN)
   si alguien la encontró: terminar
```

Comparación de tiempos según la clave (inciso 2d)

Con la estrategia cíclica y 4 procesos, la clave k le corresponde al proceso k mod p y se prueba en el intento número (k div p) + 1 de ese proceso. Esto explica las tres pruebas:

- **Clave 123456 (prueba a).** Le toca al proceso 0 (123456 mod 4 = 0), que la alcanza en su intento 30 865. Se encontró en 0.007875 s (Figura 7).
- **Clave 2^54 = 18 014 398 509 481 984 (prueba b).** Es múltiplo de 4, así que también le toca al proceso 0, pero en su intento 2^52 + 1 ≈ 4.5 × 10^15. Con el rendimiento medido de 17 063 317 claves/s (unos 4.3 × 10^6 por proceso), el tiempo estimado es de 33.45 años (Figura 9).
- **Clave 2^54 + 1 = 18 014 398 509 481 985 (prueba c).** Le toca al proceso 1 (resto 1), también en su intento ≈ 2^52, por lo que el tiempo estimado es prácticamente el mismo: unos 33.45 años.

El tiempo de búsqueda no depende de qué tan "grande" sea la clave, sino de su posición dentro del recorrido del proceso que la contiene. La clave 123456, numéricamente pequeña, y las claves 2^54 y 2^54 + 1, numéricamente enormes, solo se diferencian en qué tan pronto las alcanza algún proceso. Por eso una prueba termina en milisegundos y las otras dos son, en la práctica, inalcanzables con esta distribución. Esto motiva la estrategia alternativa de la sección 6, que reordena el recorrido para cambiar qué claves se encuentran primero.

6. Estrategia alternativa: bloques intercalados (dovetailing)

Cada proceso recibe un bloque contiguo de 2^54 claves, lo divide en K sub-bloques y avanza una clave en cada sub-bloque por turnos. Así tiene K puntos de arranque en lugar de uno.

Origen de la idea. Georg Cantor demostró que las fracciones se pueden enumerar recorriendo infinitas listas por turnos, en lugar de agotar una antes de pasar a la siguiente. En computación la técnica se llama *dovetailing*. Nosotros aplicamos ese principio sobre un número finito de sub-bloques, por lo que el recorrido diagonal de Cantor se reduce a un turno rotativo.

Por el teorema de la división euclidiana, cada t produce exactamente una clave y cada clave un único t. Ninguna clave se repite ni se omite, y el trabajo total es el mismo que en la estrategia cíclica.

Ejemplo pequeño. Con 64 claves, 4 procesos y K = 4 (bloques de 16, sub-bloques de 4):

Intento	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**…**	**16**

Proceso 0	0	4	8	12	1	5	9	13	…	15
Proceso 1	16	20	24	28	17	21	25	29	…	31
Proceso 2	32	36	40	44	33	37	41	45	…	47
Proceso 3	48	52	56	60	49	53	57	61	…	63

En los primeros 4 intentos ya se revisaron 16 puntos repartidos por todo el espacio. Con bloques simples solo se habrían revisado 0, 16, 32 y 48. Si el bloque se ve como una matriz de K filas, bloques simples lo lee por filas y dovetailing lo lee por columnas.

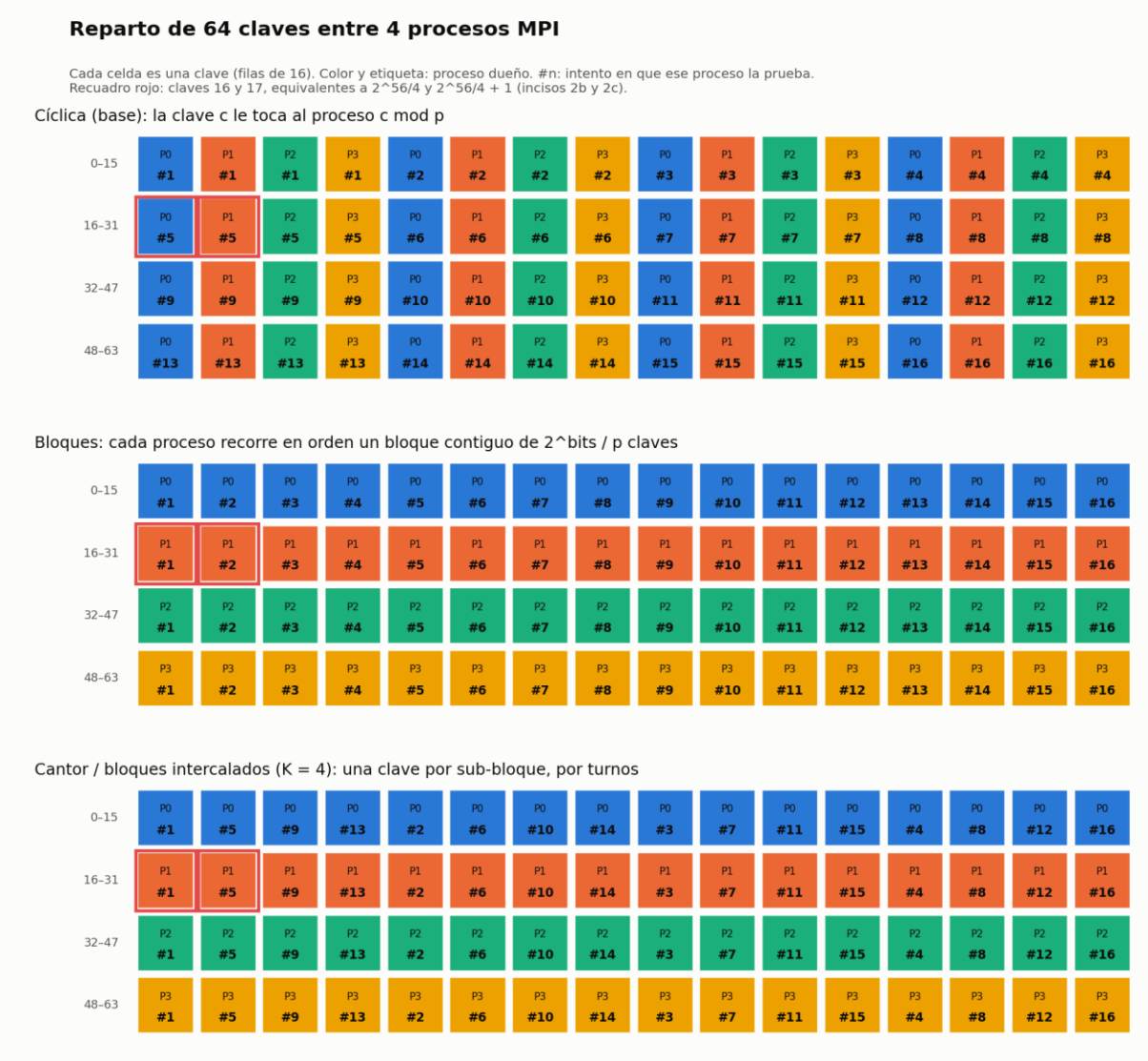


Figura 17. Reparto de claves entre procesos: cíclica vs. dovetailing.

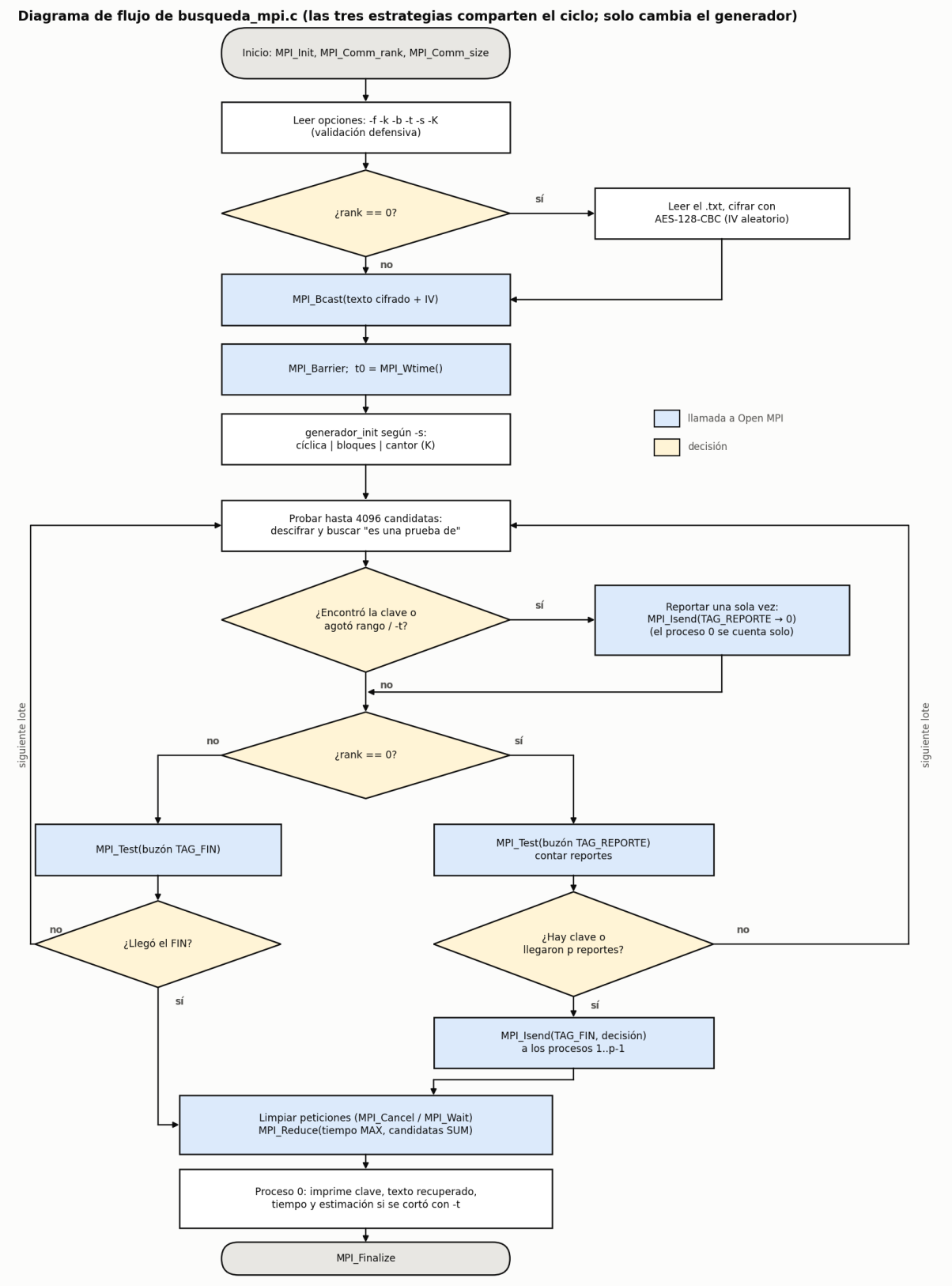


Figura 18. Diagrama de flujo de la búsqueda paralela (busqueda_mpi.c).

Comunicación entre procesos

1. El proceso 0 lee el archivo, cifra el mensaje y difunde el texto cifrado y el IV con MPI_Bcast.
2. Cada proceso abre un buzón no bloqueante con MPI_Irecv (cualquier origen, etiqueta de fin).
3. Cada proceso recorre su bloque en orden intercalado y cada N intentos revisa el buzón con MPI_Test.
4. El que encuentra la clave la envía a los demás procesos con MPI_Send y termina.
5. Los que reciben el aviso terminan. Quien no recibió nada cancela su petición con MPI_Cancel y MPI_Wait antes de MPI_Finalize.
6. El proceso 0 reúne el tiempo máximo y el total de intentos con MPI_Reduce e imprime el resultado.

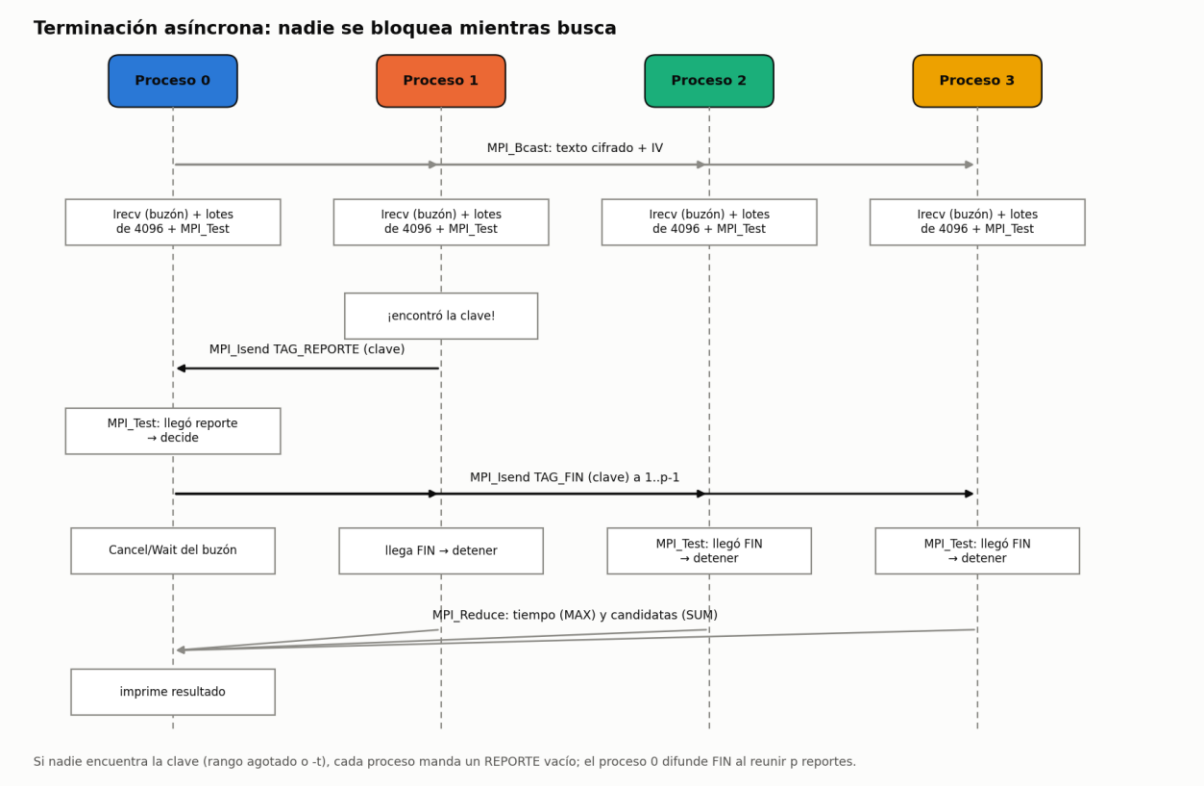


Figura 19. Comunicación entre procesos y terminación asíncrona.

Las pruebas con K = 256 mostraron el intercambio: más puntos de arranque rápidos (4·K = 1 024), pero unos 31.6 millones de intentos para 123456. El speedup frente a la versión secuencial se calculó con la clave 123456, que es la única que ambas versiones terminan en un tiempo razonable.

7. Ventajas, limitaciones y discusión

Ninguna estrategia reduce el trabajo total ni el tiempo promedio: todas prueban las mismas 2^56 claves en distinto orden. Lo que cambia es qué claves se encuentran rápido y cuánto se gasta en comunicación.

Estrategia	Ventajas	Limitaciones
Cíclica	La más rápida con claves pequeñas; reparto equilibrado	b) y c) tardan años; MPI_Allreduce obliga a todos a esperarse
Bloques (K = 1)	b) y c) instantáneas; simple de explicar	Solo 4 puntos de arranque; claves a mitad de bloque tardan años
Dovetailing (K = 16)	a), b) y c) en menos de un segundo; 64 puntos de arranque; sin trabajo extra	Claves pequeñas K veces más lentas que con bloques; el peor caso sigue en 2^54

La terminación asíncrona beneficia a las estrategias por bloques: ningún proceso se detiene a sincronizar, solo revisa un buzón. El costo es un código más cuidadoso, porque hay que cancelar las peticiones pendientes y elegir bien cada cuántos intentos se revisa el buzón.

8. Conclusiones y recomendaciones

Comprobación de la hipótesis

La hipótesis planteada en la introducción se confirma. Se propuso que una estrategia alternativa (dovetailing) reduciría el tiempo de búsqueda de las claves del enunciado y que ese tiempo depende de la posición de la clave en el recorrido de su proceso, no de su valor numérico.

Ambas afirmaciones se cumplieron. Con la estrategia cíclica, las claves b) 2^54 y c) 2^54 + 1 resultaron inalcanzables (tiempo estimado de décadas), mientras que con bloques y dovetailing se encontraron en menos de 2 milisegundos (Figura 13). La dependencia de la posición quedó demostrada: la clave 2^54, numéricamente enorme, se encuentra de inmediato porque es el inicio del bloque de un proceso, mientras que la clave 123456, numéricamente pequeña, tarda hasta ~9 s con cantor K = 256 al quedar en un intento lejano; además, la clave a mitad de bloque solo la resuelve rápido el dovetailing (0.001 s) y el peor caso (2^54−1) se agota en todas las estrategias por bloques.

El costo de reordenar el recorrido es que el trabajo total no disminuye y que las claves pequeñas se vuelven K veces más lentas: para la clave 123456 el Speedup es menor que 1 (0.71 con bloques y 0.04 con cantor K = 16, Figuras 14 y 15), es decir, la versión paralela con dovetailing es más lenta que la secuencial para esa clave. Se concluye, por tanto, que ninguna distribución estática es óptima para todas las claves: la mejor estrategia depende de qué posiciones del espacio conviene cubrir primero.

Conclusiones

- El tiempo de búsqueda depende de la posición de la clave en el recorrido de su proceso, no de su tamaño.
- El dovetailing resolvió las tres claves del enunciado en menos de un segundo sin agregar trabajo, a cambio de hacer K veces más lentas las claves pequeñas.
- La terminación asíncrona reduce el costo de comunicación frente a la sincronización colectiva periódica.
- Ninguna distribución estática es la mejor para todas las claves; la elección depende de qué posiciones conviene cubrir primero.

Recomendaciones

- Elegir la estrategia según dónde se espera la clave: cíclica o bloques (K = 1) si puede estar al inicio del recorrido de un proceso, y dovetailing con K mayor (16–256) cuando la clave puede caer a mitad de un bloque.
- Ajustar el intervalo de revisión del buzón (MPI_Test) de forma empírica: revisarlo muy seguido desperdicia tiempo en comunicación y muy esporádico retrasa la terminación cuando otro proceso ya encontró la clave.
- Para medir speedup, usar claves que ambas versiones terminen en tiempo razonable (p. ej. 123456) y promediar varias repeticiones, ya que con tiempos de milisegundos los costos fijos distorsionan la medición.
- Ejecutar siempre en la misma máquina al comparar estrategias, para que los tiempos sean comparables entre sí.
- Como trabajo futuro, un reparto dinámico (robo de trabajo) podría adaptarse a la posición real de la clave en vez de usar un reparto estático.

9. Referencias

National Institute of Standards and Technology. (2001). Advanced Encryption Standard (AES) (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

Dworkin, M. (2001). Recommendation for Block Cipher Modes of Operation: Methods and Techniques (NIST Special Publication 800-38A). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38A

OpenSSL Project. (2024). EVP Symmetric Encryption and Decryption — OpenSSL Documentation. https://docs.openssl.org/master/man7/evp/

Message Passing Interface Forum. (2023). MPI: A Message-Passing Interface Standard, Version 4.1. University of Tennessee. https://www.mpi-forum.org/docs/

Sipser, M. (2013). Introduction to the Theory of Computation (3rd ed.). Cengage Learning. [Técnica de dovetailing / enumeración diagonal de Cantor]

Anexo

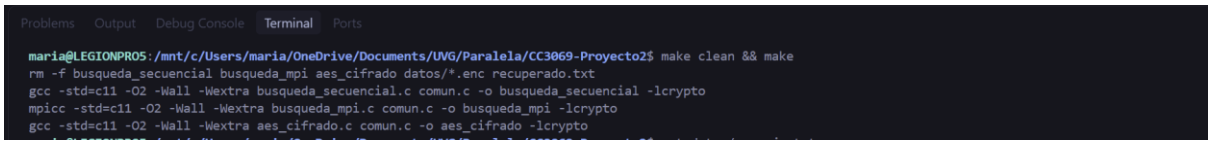


Figura 1. Compilación sin errores.



Figura 2. Texto original y cifrado del mensaje.

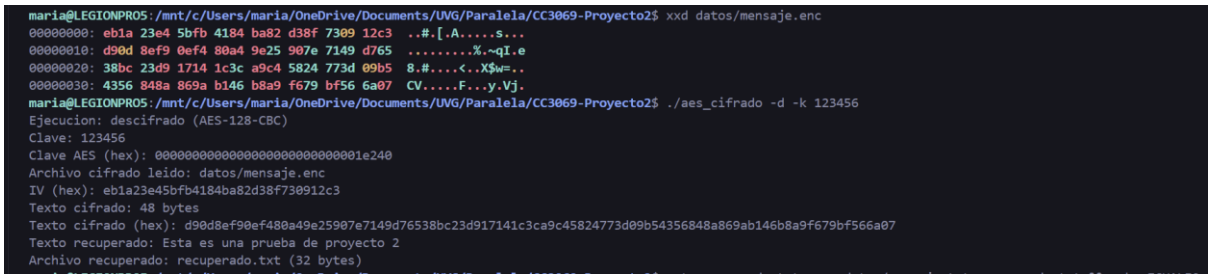


Figura 3. Bytes del archivo cifrado y descifrado del texto.



Figura 4. Ciclo completo de la solución.



Figura 5. Casos de error (clave incorrecta, archivo vacío, archivo inexistente y texto de varias líneas).

Anexo 1 — Catálogo de funciones y librerías

A1.1 Librerías utilizadas

- OpenSSL (interfaz EVP y RAND): cifrado/descifrado AES-128-CBC (EVP_aes_128_cbc, EVP_EncryptInit/Update/Final, EVP_DecryptInit/Update/Final), generación del IV aleatorio (RAND_bytes) y borrado seguro de la clave en memoria (OPENSSL_cleanse).
- Open MPI (mpi.h): paralelismo por paso de mensajes — MPI_Init/Finalize, MPI_Comm_rank/size, MPI_Bcast, MPI_Isend/MPI_Irecv/MPI_Test, MPI_Cancel/MPI_Wait, MPI_Reduce, MPI_Barrier, MPI_Wtime y MPI_Abort.
- Biblioteca estándar de C: stdio, stdlib, string, stdint/inttypes, errno, unistd (getopt para las opciones) y time (clock_gettime en la versión secuencial).

A1.2 Módulo común — comun.h / comun.c

Función	Entradas	Salida	Propósito y funcionamiento
make_key	valor (uint64), clave[16]	— (escribe clave)	Convierte el número de clave a los 16 bytes de AES. Pone los 16 bytes en cero y copia los 8 bytes de valor (big-endian) en las últimas posiciones; los primeros 8 quedan en cero.
parse_u64	cadena texto	1/0, valor por puntero	Convierte una cadena decimal a uint64 validando. Rechaza NULL, vacío o negativo; usa strtoull controlando errno y que no sobren caracteres.
leer_archivo	ruta, buffer, capacidad	resultado_lectura, leidos	Lee un .txt a memoria con validaciones. Abre en binario, detecta si el archivo excede la capacidad, quita el salto de línea final (\n o \r\n) e informa: OK, no existe, vacío, muy grande o error.
descripcion_lectura	resultado (enum)	cadena	Devuelve el mensaje legible correspondiente a un resultado_lectura.
cifrar_cbc	clave, texto, longitud	1/0, texto_cifrado	Cifra con AES-128-CBC. Deriva la clave con make_key, genera un IV aleatorio con RAND_bytes, cifra con EVP y guarda IV + datos + longitud. Borra la clave de memoria al terminar.
descifrar_cbc	clave, cifrado	1/0, texto, longitud	Operación inversa de cifrar_cbc con padding PKCS#7 por defecto; recupera el texto plano y su longitud. Borra la clave de memoria.
crear_contexto_busqueda	—	EVP_CIPHER_CTX*	Crea y preinicializa un contexto de descifrado reutilizable, para no recrearlo en cada candidata de la búsqueda.

Función	Entradas	Salida	Propósito y funcionamiento
probar_clave	ctx, candidata, cifrado	1/0/-1, plano	Prueba una clave candidata. Reinicia el contexto con la clave, desactiva el padding (la validación la hace la frase), descifra y busca la frase. Devuelve 1 si la contiene, 0 si no, −1 si falla OpenSSL.
contiene_frase	texto, longitud, frase	1/0	Indica si el texto contiene la frase. Ventana deslizante con memcmp.
imprimir_hex	etiqueta, datos, longitud	— (imprime)	Imprime una etiqueta seguida de los bytes en hexadecimal.
imprimir_duracion	etiqueta, segundos	— (imprime)	Imprime una duración con la unidad adecuada (segundos, minutos, horas, días o años).

A1.3 Cifrado/descifrado desde archivo — aes_cifrado.c

Función	Entradas	Salida	Propósito y funcionamiento
leer_opciones	argc, argv	1/0, opciones	Procesa los argumentos (-c/-d, -k, -f, -o) con getopt y valida la combinación.
escribir_archivo	ruta, datos, longitud	1/0	Escribe un buffer a un archivo en binario.
guardar_cifrado	ruta, cifrado	1/0	Guarda el IV seguido de los datos cifrados en el archivo .enc.
cargar_cifrado	ruta	1/0, cifrado	Lee de un .enc el IV y los datos cifrados a la estructura.
imprimir_encabezado / imprimir_cifrado	modo/clave, cifrado	— (imprime)	Muestran el modo y la clave, y el IV y el texto cifrado en hexadecimal.
cifrar_archivo	opciones	1/0, cifrado	Flujo completo de cifrado: lee el .txt, cifra con cifrar_cbc y guarda el .enc.
descifrar_archivo	entrada, salida, clave	1/0	Flujo de descifrado: carga el .enc, descifra con descifrar_cbc y escribe el texto recuperado.
fallar / uso	descripcion / programa	—	Reportan un error y terminan, o imprimen la ayuda de uso.
main	argc, argv	código de salida	Orquesta los modos -c (cifrar), -d (descifrar) y el ciclo completo.

A1.4 Búsqueda secuencial — busqueda_secuencial.c

Función	Entradas	Salida	Propósito y funcionamiento

segundos_actuales	—	double	Devuelve el tiempo actual con clock_gettime(CLOCK_MONOTONIC) para medir la búsqueda.
leer_opciones	argc, argv	1/0, opciones	Procesa -f, -k, -b (bits) y -t (límite de tiempo).
fallar / uso	—	—	Error con salida, o ayuda de uso.
main	argc, argv	código de salida	Cifra el mensaje, recorre el espacio de claves de forma secuencial probando cada candidata, mide el tiempo y, si se corta con -t, estima el total.

A1.5 Búsqueda paralela — busqueda_mpi.c

Función	Entradas	Salida	Propósito y funcionamiento
minimo	a, b	uint64	Mínimo de dos enteros sin signo.
inicio_parte / largo_parte	n, partes, i	uint64	Reparto casi uniforme de n elementos en partes: inicio y tamaño del bloque del proceso i (las primeras n % partes reciben uno extra).
parte_de	n, partes, x	uint64	Inversa de inicio_parte: indica en qué parte cae el elemento x.
subbloques_efectivos	estrategia, subbloques, largo	uint64	Número real de sub-bloques (K): 1 para bloques y nunca más que la cantidad de claves del bloque.
generador_init	opciones, rank, size	— (inicializa generador)	Prepara el generador de candidatas según la estrategia: paso size en cíclica, o bloque contiguo partido en K sub-bloques en bloques/cantor.
generador_siguiente	generador	1/0, salida	Entrega la siguiente candidata del proceso. En cíclica avanza de size en size; en cantor recorre el bloque por columnas (dovetailing). Devuelve 0 al agotarse.
posicion_clave	opciones, size, clave	proceso, intento	Calcula en qué proceso y en qué intento caería una clave; solo se usa para estimar tiempos (la búsqueda no conoce la clave).
nombre_estrategia	estrategia	cadena	Nombre legible de la estrategia para los mensajes.
buscar	ctx, cifrado, rank, size, opciones	clave encontrada, probadas	Bucle de búsqueda con terminación asíncrona: prueba lotes de candidatas y, con MPI_Isend/MPI_Irecv/MPI_Test, avisa y detecta cuándo algún proceso encontró la clave o se agotó el rango, cancelando las peticiones pendientes al salir.

fallar / uso / leer_opciones	—	—	Error con MPI_Abort, ayuda de uso, y procesado de -f -k -b -t -s -K.
main	argc, argv	código de salida	Inicializa MPI; el proceso 0 cifra el mensaje y lo difunde con MPI_Bcast; cada proceso busca con buscar; reúne el tiempo máximo y el total de intentos con MPI_Reduce e imprime el resultado.

Anexo 2 — Bitácora de pruebas y Speedup

Nota sobre los equipos de prueba. Las ejecuciones se realizaron en dos computadoras distintas, por lo que los tiempos estimados difieren entre bloques de figuras (por ejemplo, ≈ 33 años frente a ≈ 107 años para la clave 2^54). Las Figuras 6 a 10 corresponden a un equipo y las Figuras 11 a 16 al otro. Dentro de cada figura los tiempos sí son comparables entre sí; no así entre figuras de equipos diferentes.

Entorno de pruebas. Ubuntu sobre WSL 2, Open MPI 5.0.10, OpenSSL (interfaz EVP), compilación con gcc/mpicc -O2. Todas las búsquedas paralelas se ejecutaron con mpirun -np 4. El cifrado usa AES-128-CBC y la validación de cada candidata busca la frase “es una prueba de” en el texto descifrado. La compilación sin errores se evidencia en la Figura 6.

Resultados de las tres pruebas (4 procesos, estrategia cíclica).

Prueba	Clave	Resultado	Intento del proceso dueño	Tiempo
a)	123456	Encontrada	30 865 (proceso 0)	0.007875 s (Figura 7)
b)	18 014 398 509 481 984 (2^54)	No termina	2^52 + 1 ≈ 4.5 × 10^15 (proceso 0)	≈ 33.45 años, estimado (Figura 9)
c)	18 014 398 509 481 985 (2^54 + 1)	No termina	≈ 2^52 ≈ 4.5 × 10^15 (proceso 1)	≈ 33.45 años, estimado

Para las pruebas b) y c) se usó el límite -t 30: el programa se detiene al cumplirse el tiempo y estima el total a partir del rendimiento medido (17 063 317 claves/s con 4 procesos) y de la posición de la clave en el recorrido de su proceso (Figura 9).

Speedup con la clave 123456. Promedio de 10 repeticiones de la versión secuencial frente a 2, 3 y 4 procesos, obtenido con ./medir_speedup.sh 123456 56 10 (Figura 8).

Procesos	Tiempo prom. (s)	Speedup	Eficiencia

1 (secuencial)	0.044610	1.00	100.0 %
2	0.015322	2.91	145.6 %
3	0.010751	4.15	138.3 %
4	0.007851	5.68	142.1 %

Speedup = T_secuencial / T_paralelo; Eficiencia = Speedup / número de procesos.

Análisis del Speedup. El Speedup crece con el número de procesos y resulta superlineal en todos los casos (5.68× con 4 procesos, eficiencias entre 138 % y 146 %). La distribución cíclica explica un Speedup cercano a p: con p procesos, el dueño de la clave 123456 solo necesita alrededor de 123 457 / p intentos (30 865 con 4 procesos), mientras que la versión secuencial prueba 123 457 candidatas. El trabajo total no disminuye; de hecho, la versión paralela revisa algunas candidatas de más (129 169 en la Figura 7) porque la terminación se comprueba cada 512 intentos. El excedente sobre p se debe a que la versión secuencial procesa menos claves por segundo (≈ 2.8 × 10^6) que cada proceso MPI (≈ 4.1 × 10^6). Con tiempos de milisegundos, los costos fijos (inicialización de OpenSSL, calentamiento de caché y frecuencia del procesador) pesan mucho en la medición, por lo que el resultado no debe interpretarse como una aceleración “gratis”; con claves más lejanas se esperaría una eficiencia más cercana al 100 %.

Programación defensiva. El programa maneja entradas inválidas con mensajes claros y código de salida distinto de cero (Figura 10): archivo inexistente, archivo vacío y clave no numérica, mostrando además la ayuda de uso con las opciones -f, -k, -b y -t.

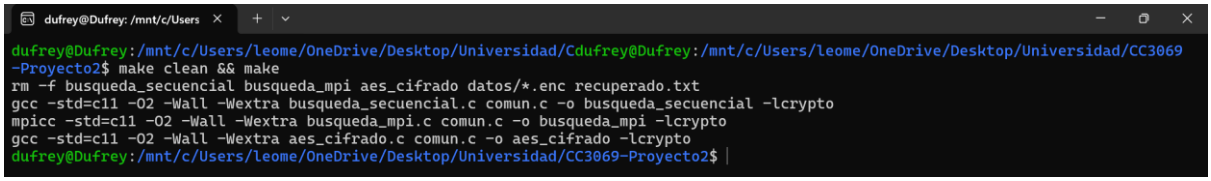


Figura 6. Compilación del proyecto sin errores (make clean && make).



Figura 7. Búsqueda de la clave 123456 con 4 procesos MPI (estrategia cíclica).

```
dufrey@Dufrey:/mnt/c/Users/leome/OneDrive/Desktop/Universidad/CC3069-Proyecto2$ ./medir_speedup.sh 123456 56 10
Escenario: clave 123456, 2^56 candidatas, 10 repeticiones

Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos   | Tiempo prom (s) | Speedup | Eficiencia |
|------------|-----------------|---------|------------|
| 1 (sec.)   |        0.044610 |    1.00 |     100.0% |
| 2          |        0.015322 |    2.91 |     145.6% |
| 3          |        0.010751 |    4.15 |     138.3% |
| 4          |        0.007851 |    5.68 |     142.1% |
```

Figura 8. Tabla de Speedup con la clave 123456 (medir_speedup.sh 123456 56 10).

```
dufrey@Dufrey:/mnt/c/Users/leome/OneDrive/Desktop/Universidad/CC3069-Proyecto2$ mpirun -np 4 ./busqueda_mpi -k 18014398509481984 -t
 30
Ejecucion: paralela (Open MPI, 4 procesos, estrategia ciclica)
Archivo: datos/mensaje.txt (32 bytes)
Clave secreta: 18014398509481984
IV (hex): 8252716f12fa28bf9c38cbbb8a7ca267
Texto cifrado (hex): f55b9ab6fa20cb7d86a181f8f0245022a249b82d117ca9fdd3950adab58fd6675dd27d30924016263 88f64ac1488cf24
Frase de busqueda: "es una prueba de"
Rango explorado: 2^56 = 72057594037927936 candidatas
Resultado: no se encontro la clave (rango agotado o limite de tiempo)
Candidatas probadas: 511901696
Tiempo: 30.000128 segundos
Rendimiento: 17063317 claves/s
Intentos del proceso dueno: 4503599627370497
Tiempo estimado: 33.45 años
```

Figura 9. Búsqueda de la clave 2^54 con límite de 30 s y tiempo estimado.

```
dufrey@Dufrey:/mnt/c/Users/leome/OneDrive/Desktop/Universidad/CC3069-Proyecto2$ ./busqueda_secuencial -f datos/no_existe.txt
./busqueda_secuencial -f datos/vacio.txt
./busqueda_secuencial -k abc
Error con 'datos/no_existe.txt': el archivo no existe.
Error con 'datos/vacio.txt': el archivo esta vacio.
Uso: ./busqueda_secuencial [-f archivo] [-k clave] [-b bits] [-t segundos]
  -f archivo   texto a cifrar (por defecto datos/mensaje.txt)
  -k clave     clave secreta en decimal (por defecto 123456)
  -b bits      explora 2^bits candidatas (1-56, por defecto 56)
  -t segundos  limite de tiempo; al cumplirse se estima el total
```

Figura 10. Programación defensiva: archivo inexistente, archivo vacío y clave no numérica.

```
$ ./busqueda_secuencial -k 123456
Ejecucion: secuencial (AES-128-CBC)
Archivo: datos/mensaje.txt (32 bytes)
Texto original: Esta es una prueba de proyecto 2
Clave secreta: 123456
Clave AES (hex): 0000000000000000000000000001e240
IV (hex): 95a0967c7ec90069e4fad14c29efb218
Texto cifrado (hex): 4ef67d7da9876a86a290d2d6a8fbf4f64e785ca1d21bbebc21e221dded1c856fddff6a5ebb17c0c525198c4ada4ae0ac
Frase de busqueda: "es una prueba de"
Rango explorado: 2^56 = 72057594037927936 candidatas
Resultado: ENCONTRADA
Clave encontrada: 123456
Texto recuperado: Esta es una prueba de proyecto 2
Verificacion: coincide con la clave usada para cifrar
Candidatas probadas: 123457
Tiempo: 0.033066 segundos
Rendimiento: 3733699 claves/s

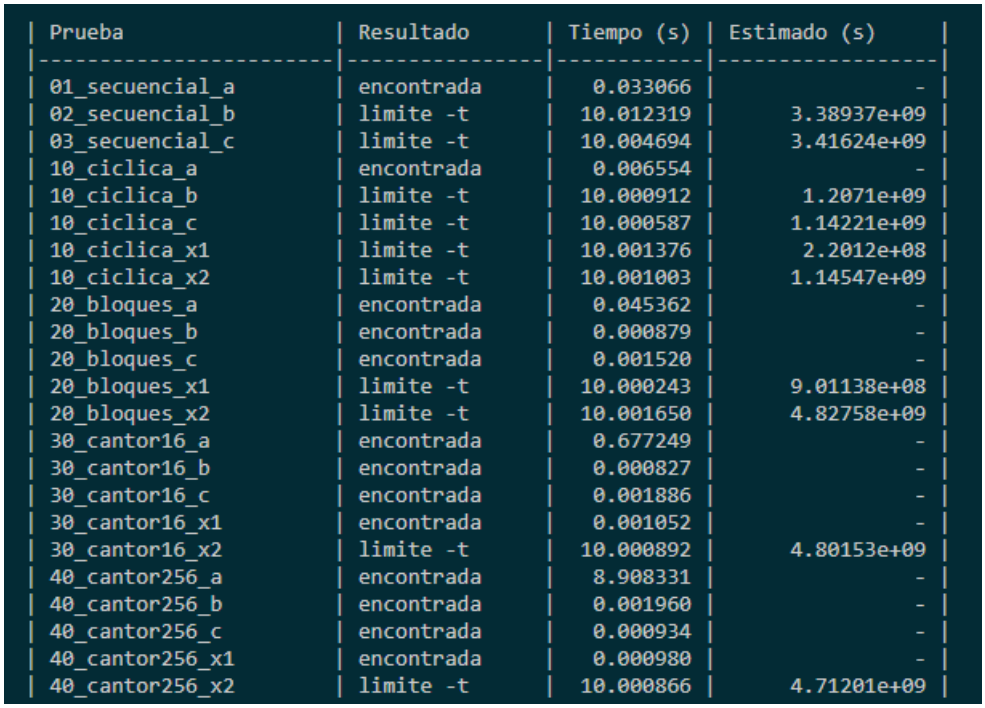
$ ./busqueda_secuencial -k 18014398509481984 -t 10
Ejecucion: secuencial (AES-128-CBC)
Archivo: datos/mensaje.txt (32 bytes)
Texto original: Esta es una prueba de proyecto 2
Clave secreta: 18014398509481984
Clave AES (hex): 00000000000000000040000000000000
IV (hex): 36f21a2645dee0c8080ab7ef81a91eb4
Texto cifrado (hex): 99c29226fb15dd54ad0ba99ed76f357324c36435646594107f08cfecc288ae4a1a70a111f586e197d429f4eb977c620d
Frase de busqueda: "es una prueba de"
Rango explorado: 2^56 = 72057594037927936 candidatas
Resultado: LIMITE (se detuvo a los 10 s)
Candidatas probadas: 53215232
Tiempo: 10.012319 segundos
Rendimiento: 5314975 claves/s
Intentos necesarios: 18014398509481984
Tiempo estimado hasta la clave (s): 3.38937e+09
Tiempo estimado hasta la clave: 107.40 años
```

Figura 11. Versión secuencial (AES-128-CBC): clave 123456 encontrada en 0.033 s y clave 2^{54} cortada a los 10 s con su tiempo estimado (≈107 años).

```
$ mpirun -np 4 ./busqueda_mpi -k 123456 -s ciclica -t 10
Ejecucion: paralela (Open MPI, 4 procesos, estrategia ciclica)
Archivo: datos/mensaje.txt (32 bytes)
Clave secreta: 123456
IV (hex): a9fcc52b0c5d321a59c66a1a84893423
Texto cifrado (hex): cbee0c8fb40d6c830a9049534d78b3940a8bca8c778498af4a8e89109efece0fdf20644b3d8031e8bcd74d1bf6a97dc3
Frase de busqueda: "es una prueba de"
Rango explorado: 2^56 = 72057594037927936 candidatas
Posicion de la clave: proceso 0, intento 30865
Resultado: ENCONTRADA
Clave encontrada: 123456
Texto recuperado: Esta es una prueba de proyecto 2
Verificacion: coincide con la clave usada para cifrar
Candidatas probadas: 129169
Tiempo: 0.006554 segundos
Rendimiento: 19708714 claves/s

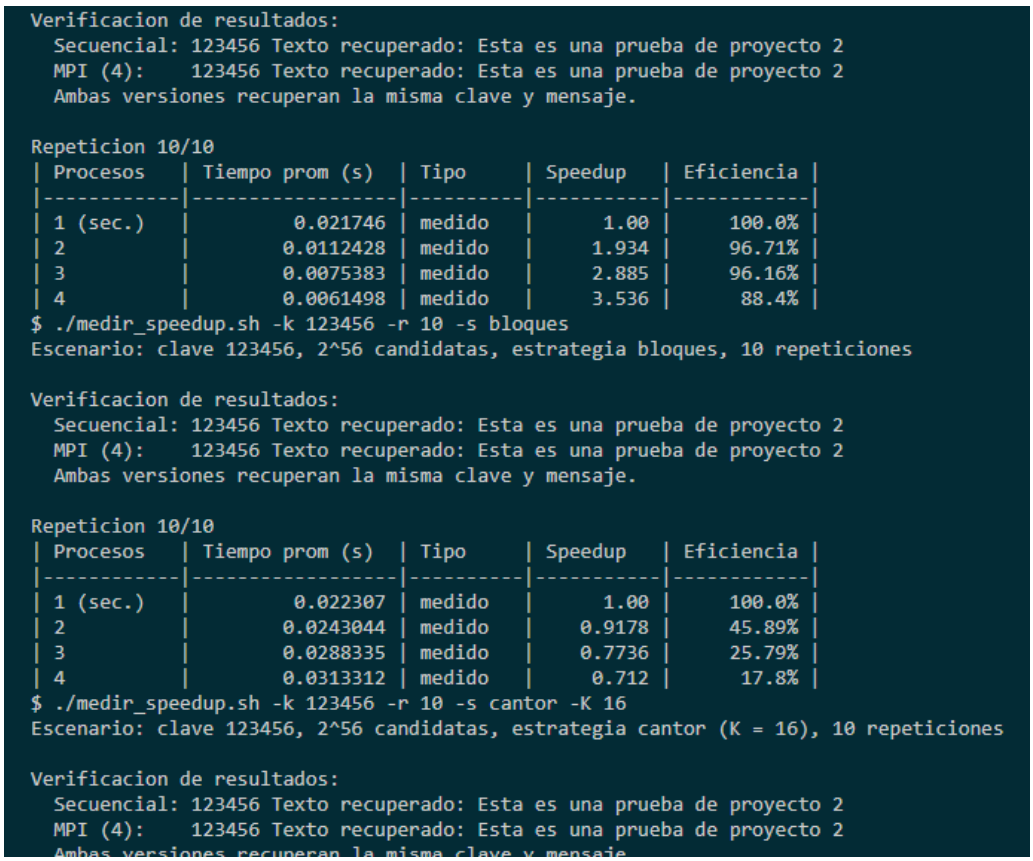
$ mpirun -np 4 ./busqueda_mpi -k 123456 -s bloques -t 10
Ejecucion: paralela (Open MPI, 4 procesos, estrategia bloques)
Archivo: datos/mensaje.txt (32 bytes)
Clave secreta: 123456
IV (hex): 3f65b4ddff5265cadd9c092b9558d57e
Texto cifrado (hex): e53b24e693850eb5cb1239261d5111685db68d1a2855bb8f0854f3affd0f3a703c3a1e9623e0e78f9548b70e2637f928
Frase de busqueda: "es una prueba de"
Rango explorado: 2^56 = 72057594037927936 candidatas
Posicion de la clave: proceso 0, intento 123457
Resultado: ENCONTRADA
Clave encontrada: 123456
Texto recuperado: Esta es una prueba de proyecto 2
Verificacion: coincide con la clave usada para cifrar
Candidatas probadas: 492097
Tiempo: 0.045362 segundos
Rendimiento: 10848265 claves/s
```

Figura 12. Búsqueda paralela de la clave 123456 con 4 procesos: estrategia cíclica (encontrada en el intento 30 865 del proceso 0) y estrategia por bloques (encontrada).



Prueba	Resultado	Tiempo (s)	Estimado (s)
01_secuencial_a	encontrada	0.033066	-
02_secuencial_b	limite -t	10.012319	3.38937e+09
03_secuencial_c	limite -t	10.004694	3.41624e+09
10_ciclica_a	encontrada	0.006554	-
10_ciclica_b	limite -t	10.000912	1.2071e+09
10_ciclica_c	limite -t	10.000587	1.14221e+09
10_ciclica_x1	limite -t	10.001376	2.2012e+08
10_ciclica_x2	limite -t	10.001003	1.14547e+09
20_bloques_a	encontrada	0.045362	-
20_bloques_b	encontrada	0.000879	-
20_bloques_c	encontrada	0.001520	-
20_bloques_x1	limite -t	10.000243	9.01138e+08
20_bloques_x2	limite -t	10.001650	4.82758e+09
30_cantor16_a	encontrada	0.677249	-
30_cantor16_b	encontrada	0.000827	-
30_cantor16_c	encontrada	0.001886	-
30_cantor16_x1	encontrada	0.001052	-
30_cantor16_x2	limite -t	10.000892	4.80153e+09
40_cantor256_a	encontrada	8.908331	-
40_cantor256_b	encontrada	0.001960	-
40_cantor256_c	encontrada	0.000934	-
40_cantor256_x1	encontrada	0.000980	-
40_cantor256_x2	limite -t	10.000866	4.71201e+09

Figura 13. Bitácora de ejecuciones individuales (./bitacora.sh): resultado, tiempo y tiempo estimado de las cinco claves con las estrategias cíclica, bloques, cantor K=16 y cantor K=256.



```
Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |         0.021746 | medido   |      1.00 |     100.0% |
| 2          |        0.0112428 | medido   |     1.934 |     96.71% |
| 3          |        0.0075383 | medido   |     2.885 |     96.16% |
| 4          |        0.0061498 | medido   |     3.536 |      88.4% |
$ ./medir_speedup.sh -k 123456 -r 10 -s bloques
Escenario: clave 123456, 2^56 candidatas, estrategia bloques, 10 repeticiones

Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |         0.022307 | medido   |      1.00 |     100.0% |
| 2          |        0.0243044 | medido   |    0.9178 |     45.89% |
| 3          |        0.0288335 | medido   |    0.7736 |     25.79% |
| 4          |        0.0313312 | medido   |     0.712 |      17.8% |
$ ./medir_speedup.sh -k 123456 -r 10 -s cantor -K 16
Escenario: clave 123456, 2^56 candidatas, estrategia cantor (K = 16), 10 repeticiones

Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje
```

Figura 14. Tablas de speedup con la clave 123456 (10 repeticiones): estrategias cíclica y por bloques.

```
Repeticion 10/10
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |         0.022307 | medido   |      1.00 |     100.0% |
| 2          |        0.0243044 | medido   |    0.9178 |     45.89% |
| 3          |        0.0288335 | medido   |    0.7736 |     25.79% |
| 4          |        0.0313312 | medido   |     0.712 |      17.8% |
$ ./medir_speedup.sh -k 123456 -r 10 -s cantor -K 16
Escenario: clave 123456, 2^56 candidatas, estrategia cantor (K = 16), 10 repeticiones

Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje.

Repeticion 10/10
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |         0.022676 | medido   |      1.00 |     100.0% |
| 2          |         0.470125 | medido   |   0.04823 |     2.412% |
| 3          |         0.474931 | medido   |   0.04775 |     1.592% |
| 4          |         0.534941 | medido   |   0.04239 |      1.06% |
$ ./medir_speedup.sh -k 123456 -r 5 -s cantor -K 256
Escenario: clave 123456, 2^56 candidatas, estrategia cantor (K = 256), 5 repeticiones

Verificacion de resultados:
  Secuencial: 123456 Texto recuperado: Esta es una prueba de proyecto 2
  MPI (4):    123456 Texto recuperado: Esta es una prueba de proyecto 2
  Ambas versiones recuperan la misma clave y mensaje.
```

Figura 15. Tablas de speedup con la clave 123456: estrategia cantor K=16 (10 rep.) y cantor K=256 (5 rep.)

```
Repeticion 5/5
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |         0.031099 | medido   |      1.00 |     100.0% |
| 2          |          7.68726 | medido   |  0.004046 |    0.2023% |
| 3          |          8.23138 | medido   |  0.003778 |    0.1259% |
| 4          |          8.64033 | medido   |  0.003599 |   0.08998% |
$ ./medir_speedup.sh -k 18014398509481984 -r 1 -s ciclica -t 10
Escenario: clave 18014398509481984, 2^56 candidatas, estrategia ciclica, 1 repeticiones, limite 10 s

Verificacion de resultados:
  Secuencial:
  MPI (4):
  Al menos una version no termino; se comparan tiempos estimados.

Repeticion 1/1
| Procesos   | Tiempo prom (s)  | Tipo     | Speedup   | Eficiencia |
|------------|------------------|----------|-----------|------------|
| 1 (sec.)   |      3.32585e+09 | estimado |      1.00 |     100.0% |
| 2          |      1.93722e+09 | estimado |     1.717 |     85.84% |
| 3          |      1.37026e+09 | estimado |     2.427 |     80.91% |
| 4          |      1.10291e+09 | estimado |     3.016 |     75.39% |
$ ./medir_speedup.sh -k 18014398509481984 -r 3 -s cantor -K 16 -t 10
Escenario: clave 18014398509481984, 2^56 candidatas, estrategia cantor (K = 16), 3 repeticiones, limite 10 s

Verificacion de resultados:
  Secuencial:
  MPI (4):    18014398509481984 Texto recuperado: Esta es una prueba de proyecto 2
  Al menos una version no termino; se comparan tiempos estimados.
```

Figura 16. Tablas de speedup con la clave 2^{54} (límite 10 s, tiempos estimados): estrategias cíclica y cantor K=16.